TradeDocs
Production Build Plan

Shipment Workspace + Reusable Trade Document Sets

COMMERCIAL PRODUCTION-READY SPECIFICATION

Version	1.0
Date	6 September 2026
Audience	Codex, Claude Code, engineering reviewers and product owner
Delivery standard	Deployable, saleable and operable - not an MVP

Final outcome

A complete commercial product ready to publish, sell, monitor and maintain after launch. Human approvals remain mandatory for production credentials, legal/regulatory sign-off and go-live.

How to Use This Specification

Give this entire PDF to the coding agent. The agent must execute the numbered tasks as delivery contracts. Common Tasks 01-10 establish the platform; product tasks continue the numbering. A task is complete only when every acceptance criterion and required test has evidence.

Document conventions

- MUST / SHALL = required for launch.
- SHOULD = expected unless a documented ADR explains why not.
- MAY = optional and must not displace required work.
- Human approval = a named product/legal/operations decision recorded in the repository; an AI self-approval is not sufficient.
- Production-ready = functional product plus security, privacy, accessibility, billing integrity, observability, backups, support and regulatory maintenance.

Recommended repository documents

- PRODUCT.md and ROADMAP.md: product boundaries, personas, pricing decisions and success measures.
- ARCHITECTURE.md and DATABASE.md: diagrams, trust boundaries, schema, RLS and data lifecycle.
- SECURITY.md, LEGAL.md and regulatory registry: threat model, policies, sources, approvals and limitations.
- SEO.md and ANALYTICS.md: route inventory, indexation rules, event catalog and dashboards.
- OPERATIONS.md, RUNBOOK.md and TEST_PLAN.md: SLOs, alerts, backups, incidents, release gates and evidence.
- AGENTS.md: execution rules from this specification.

Product Definition

Product promise

A commercial shipment workspace that captures company, customer and product data once and reuses it to create coherent trade-document sets, calculators and repeatable shipment history.

Primary personas

Small exporters/importers, manufacturers, wholesalers, freight operations teams, trade consultants preparing drafts, and internal support/compliance operators.

Explicit non-goals and legal boundary

Not a customs broker, carrier, chamber of commerce, issuing authority, legal adviser, sanctions-screening service, tariff classifier, or source of negotiable transport documents unless separately authorized and validated.

Launch success conditions

- A returning user creates a shipment primarily from saved master data and generates a consistent set.
- Every finalized document is reproducible, immutable and traceable to a shipment revision.
- Certificate/BOL-style outputs cannot be mistaken for official or carrier-issued documents.
- Free tools acquire traffic and convert it into saved workspace reuse and purchases.
- Operations can reconcile, support, restore and update the product safely.

Core data inventory

profiles, organizations, memberships, companies, contacts, parties, addresses, products, product_variants, shipments, shipment_items, packages, package_items, transport_legs, document_sets, documents, document_revisions, templates, attachments, render_jobs, numbering_sequences, orders, payments, entitlements, email_events, audit_events, regulatory_sources, feature_flags

Required stack

- Next.js App Router + strict TypeScript; accessible component system and responsive CSS.
- Supabase Postgres, Auth and private Storage with migrations, generated types and RLS.
- Provider-neutral Payment Links adapter with verified webhooks and entitlements.
- Vercel preview/staging/production delivery; Resend + React Email.
- Server-side PDF/document engine with versioned renderers and visual regression.
- GA4, Google Search Console and IndexNow API.
- Sentry, Cloudflare Turnstile, distributed rate limiting, structured logs and health checks.
- GitHub CI/CD, automated testing, backup/restore and operational runbooks.

Master Instructions for the AI Agent

These instructions override any temptation to reduce scope for speed or token economy.

- This specification defines a commercial, production-ready product. Do not reinterpret it as an MVP, prototype, demo, hackathon build, or single-page generator.
- Execute tasks in order unless a dependency graph proves safe parallelism. Do not silently omit, defer, stub, or replace a requirement. Record approved deviations in DECISIONS.md with rationale and impact.
- Before coding, inspect the repository and existing changes. Preserve user work. Maintain PRODUCT.md, ARCHITECTURE.md, DATABASE.md, SECURITY.md, LEGAL.md, SEO.md, ANALYTICS.md, DESIGN_SYSTEM.md, OPERATIONS.md, TEST_PLAN.md, RUNBOOK.md, CHANGELOG.md, and AGENTS.md as living documents.
- For every task: restate the acceptance criteria, implement the smallest complete production slice, add or update tests, run relevant checks, inspect the user-visible result, update documentation, then create one focused commit. Never claim completion while tests are skipped or failing.
- Mandatory gates for every feature: formatting, lint, TypeScript strict typecheck, unit tests, integration tests, relevant Playwright E2E, accessibility checks, and a production build. PDF or visual work also requires rendered visual inspection and regression fixtures.
- Never weaken RLS, authentication, webhook verification, rate limits, audit logging, validation, or privacy controls to make a test pass. Never expose service-role keys, payment secrets, Resend keys, Turnstile secrets, Sentry auth tokens, or private user data to the browser or repository.
- Use official primary sources for tax, payroll, customs, trade, sanctions, privacy, accessibility, payment, and platform rules. Capture source URL, jurisdiction, effective date, retrieval date, reviewer, and affected version. If legal or regulatory certainty is missing, block the affected claim or calculation behind a feature flag and present a clear limitation.
- No document may claim government, carrier, chamber, customs, tax authority, or employer endorsement unless such endorsement actually occurred. Do not market generated documents as proof of income, identity, origin, customs clearance, transport title, or legal compliance.
- Use migrations, seeds, and typed database access. Every exposed Supabase table/view/function requires an explicit access decision and RLS tests. Cross-tenant isolation failures are release blockers.
- All external callbacks and asynchronous jobs must be authenticated where applicable, idempotent, observable, retry-safe, and protected against replay. User-visible actions need loading, success, empty, and recoverable error states.
- Use feature flags and staged rollout for regulatory or high-risk changes. Production deployment requires an approved checklist, backup confirmation, rollback plan, migration plan, monitoring, and post-deploy smoke tests.
- Do not invent production credentials, tax rules, official identifiers, legal text, prices, or provider behavior. Add configuration placeholders and document the exact human decision required.

Non-Functional Release Requirements

- Availability target: 99.9% monthly for authenticated and purchase flows, excluding scheduled maintenance; define SLOs and error budgets.
- Performance budgets: public pages target LCP <= 2.5 s, INP <= 200 ms, CLS <= 0.1 at the 75th percentile; server actions and API routes publish p50/p95 latency metrics.
- Accessibility: WCAG 2.2 AA target, keyboard-only operation, semantic headings, labeled fields, visible focus, contrast compliance, reduced-motion support, and accessible PDF/document alternatives where feasible.
- Compatibility: current and previous major versions of Chrome, Safari, Firefox, and Edge; responsive behavior from 360 px mobile through large desktop; print-safe templates.
- Privacy: data minimization, purpose limitation, retention schedules, export/delete workflows, consent-aware analytics, redacted logs, and documented subprocessors.
- Recovery: daily automated backups, tested point-in-time recovery where plan permits, quarterly restore drills, RPO/RTO documented per service, and versioned storage retention.

Global definition of done

- Acceptance criteria for the active task are mapped to evidence.
- No skipped/failing checks; flaky tests are fixed or quarantined only with owner and expiry.
- Relevant UI and generated output have been inspected, not merely compiled.
- Security/privacy/accessibility/regulatory impact is reviewed.
- Documentation, migrations, rollback notes and monitoring are current.
- One focused commit exists with a useful message; repository is left clean except pre-existing user changes.

Ordered Task Index

ID	Task	Dependency phase
01	Repository foundation and decision records	Platform
02	Architecture, environments and configuration	Platform
03	Design system and application shell	Platform
04	Identity, organizations and lifecycle	Platform
05	Security, abuse prevention and auditability	Platform
06	Billing, payment links and entitlements	Platform
07	Transactional email and notification controls	Platform
08	Analytics, consent and funnel measurement	Platform
09	SEO platform and indexation controls	Platform
10	CI/CD, release safety and supply chain	Platform
11	TradeDocs domain model and database	TradeDocs
12	Reusable company, customer and product workspace	TradeDocs
13	Shipment workspace and document dependency graph	TradeDocs
14	Trade document rules and canonical schemas	TradeDocs
15	Commercial and sales document workflows	TradeDocs
16	Packing, delivery and shipping-style workflows	TradeDocs
17	Certificate of Origin preparation workflow	TradeDocs
18	Trade PDF engine, templates and branding	TradeDocs
19	Document sets, bulk operations and exports	TradeDocs
20	TradeDocs dashboard, history and collaboration	TradeDocs
21	Trade calculators, free generators and acquisition	TradeDocs
22	TradeDocs SEO and editorial system	TradeDocs
23	TradeDocs admin, support and operational controls	TradeDocs
24	Trade regulatory, terminology and source registry	TradeDocs
25	TradeDocs launch, monitoring and post-launch optimization	TradeDocs

Task 01 - Repository foundation and decision records

DELIVERY CONTRACT

Objective

Create an auditable baseline that can sustain a production SaaS through launch and maintenance.

Scope

- Separate repository, environments, domain and vendor projects for this product.
- App Router structure, strict TypeScript, package conventions, dependency policy and generated-client policy.
- Living product/architecture/security/legal/SEO/analytics/operations/test documents and ADR template.

Implementation Requirements

- Initialize Next.js with strict mode, formatting, lint rules, Tailwind and an accessible component foundation.
- Pin runtimes and package manager; configure dependency updates, license review and secret scanning.
- Define branch protection, CODEOWNERS-equivalent ownership, conventional commits, pull-request template and release tags.
- Provide .env.example containing names and purpose only, never values.

Dependencies

- None.

Acceptance Criteria

- Fresh setup reproduces locally from documented steps.
- All required living documents exist with owners and review triggers.
- Build, lint, typecheck and a smoke test pass in a clean checkout.
- Repository contains no credential or copied competitor asset.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Scan tracked files and git history for secrets; validate env schema fails fast with actionable errors.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Do not build feature pages in this task. Establish the contract future tasks must follow.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 02 - Architecture, environments and configuration

DELIVERY CONTRACT

Objective

Define the production topology and typed boundaries before feature implementation.

Scope

- Local, test, preview/staging and production environments.
- Next.js server/client boundaries; Supabase Auth/Postgres/Storage; payment adapter; Resend; PDF worker; analytics; Sentry; rate limiter; Turnstile; IndexNow.
- Configuration validation, feature flags, queues/jobs and provider failure modes.

Implementation Requirements

- Produce context/container diagrams and request/data flows.
- Use server-only modules for secrets and privileged clients.
- Define data classification, retention class, residency assumptions and ownership for every store.
- Create health/readiness endpoints that disclose no secrets.

Dependencies

- Task 01.

Acceptance Criteria

- Architecture documents map every required service and trust boundary.
- Preview cannot access production data or credentials.
- Misconfiguration fails during build/startup with a clear message.
- Vendor outage/degradation behavior is documented.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Configuration matrix tests; server-only import boundary test; health endpoint tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use provider-neutral interfaces where the specification calls for portability; avoid premature microservices.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 03 - Design system and application shell

DELIVERY CONTRACT

Objective

Deliver a credible, accessible visual system for public, generator and dashboard experiences.

Scope

- Typography, spacing, color, iconography, grids, forms, tables, dialogs, menus, tabs, toasts, skeletons, error and empty states.
- Public header/footer, mobile navigation, authenticated sidebar/topbar, command/search pattern and print styles.
- Trust, security and legal disclosure components.

Implementation Requirements

- Build reusable components with documented variants and states.
- Avoid generic AI-template aesthetics and copied competitor styling.
- Support long labels, localization-ready layout, reduced motion and keyboard navigation.
- Add a component showcase route restricted outside production or protected.

Dependencies

- Tasks 01-02.

Acceptance Criteria

- All foundational components render at mobile/tablet/desktop breakpoints.
- Keyboard and screen-reader labels work; focus never becomes trapped.
- No layout shift from icons/fonts; print styles hide navigation.
- Design decisions and tokens are documented.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Automated axe tests; visual snapshots for core states and three viewports; keyboard navigation E2E.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Visually inspect every component state, including errors and long content, before completion.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 04 - Identity, organizations and lifecycle

DELIVERY CONTRACT

Objective

Implement secure accounts and multi-tenant organizations without locking the product to individual users.

Scope

- Sign-up, email verification, sign-in, magic-link option, password reset, session management and logout-all-devices.
- Profiles, organizations, memberships, invitations, roles, company profile, data export and account deletion.
- Reauthentication for sensitive actions and lifecycle emails.

Implementation Requirements

- Use Supabase Auth and public profile tables linked by stable IDs.
- Define owner/admin/member roles and least-privilege authorization helpers.
- RLS on every tenant table; invitation expiration and single use.
- Deletion is queued, auditable, revocable during a defined grace period, then purges or anonymizes by retention policy.

Dependencies

- Tasks 01-03.

Acceptance Criteria

- A user can complete all identity recovery and organization flows.
- User A cannot enumerate/read/write User B or Organization B data.
- Last owner cannot accidentally orphan an organization.
- Export and deletion produce logged, privacy-safe outcomes.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- RLS matrix across anonymous/member/admin/owner/service roles; session fixation and invitation replay tests; deletion/export job tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Treat any cross-tenant access as a stop-ship defect; do not solve authorization only in UI code.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 05 - Security, abuse prevention and auditability

DELIVERY CONTRACT

Objective

Create defense-in-depth appropriate for financial and trade document generation.

Scope

- Threat model, secure headers, CSP, CSRF posture, SSRF defenses, upload safety, input/output encoding, bot protection and rate limits.
- Turnstile on risky anonymous flows; IP/account/device-aware quotas with privacy controls.
- Append-only audit events for authentication, data changes, generation, downloads, payments, admin actions and regulatory changes.

Implementation Requirements

- Validate all inputs server-side with shared schemas and bounded payloads.
- Use signed short-lived download links; never expose storage buckets publicly by accident.
- Redact PII and secrets in logs/Sentry; define audit retention and privileged access.
- Document incident severity, triage, notification and evidence-preservation flow.

Dependencies

- Tasks 02 and 04.

Acceptance Criteria

- Threat model covers assets, actors, abuse cases and mitigations.
- Rate limits return usable errors and cannot be bypassed through alternate routes.
- Audit events identify actor, tenant, action, target, time, correlation ID and safe metadata.
- Security headers score is validated in staging.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- OWASP-oriented abuse tests; IDOR/BOLA matrix; rate-limit concurrency tests; malicious upload and log-redaction tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Never store raw sensitive document contents in analytics, logs or error breadcrumbs.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 06 - Billing, payment links and entitlements

DELIVERY CONTRACT

Objective

Turn payment links into a trustworthy, provider-neutral commercial access layer.

Scope

- Products/plans/prices, one-time and subscription-ready entitlements, orders, payments, refunds, coupons and usage events.
- Checkout creation/redirect, success/cancel states, billing history, receipts/invoice links and support-facing lookup.
- Verified webhooks, event ledger, replay, reconciliation and dispute/refund states.

Implementation Requirements

- A success redirect is never proof of payment; only verified server-side webhook state grants entitlement.
- Store money in minor units with ISO currency and immutable price snapshots.
- Webhook handlers verify signature and timestamp, persist raw event safely, deduplicate, transact state changes and support replay.
- Daily reconciliation flags provider/database mismatches.

Dependencies

- Tasks 02, 04 and 05.

Acceptance Criteria

- Duplicate/out-of-order webhooks never duplicate access or orders.
- Entitlement checks occur server-side at every paid capability.
- Refund/dispute state removes or adjusts access according to policy without deleting customer history.
- Finance/support can trace a payment end-to-end by correlation IDs.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Signed/invalid/replayed/out-of-order webhook fixtures; currency rounding; reconciliation; refund/dispute; concurrent checkout tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Keep provider-specific code behind an adapter and write contract tests against the chosen provider sandbox.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 07 - Transactional email and notification controls

DELIVERY CONTRACT

Objective

Deliver reliable, branded, privacy-safe lifecycle and document emails.

Scope

- Resend + React Email templates for verification, access recovery, invitation, purchase, receipt, document delivery, job result and security alerts.
- Sending domains, SPF/DKIM/DMARC checklist, suppression/bounce/complaint webhooks, retry policy and notification preferences.
- Email-safe download links and support traceability.

Implementation Requirements

- Use versioned templates and stable event IDs.
- Do not attach sensitive PDFs by default; prefer authenticated or short-lived signed links.
- Persist delivery state without storing unnecessary email bodies.
- Separate transactional mail from future marketing consent.

Dependencies

- Tasks 04-06.

Acceptance Criteria

- Every required event has a branded HTML and plain-text rendering.
- Duplicate jobs cannot send duplicate purchase or delivery messages.
- Bounces/complaints suppress future eligible sends and surface to support.
- Links expire and cannot expose another tenant's document.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Template snapshot and accessibility tests; webhook verification; retry/idempotency; expired-link and cross-tenant tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use sandbox recipients in non-production and prevent preview deployments from mailing real customers.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 08 - Analytics, consent and funnel measurement

DELIVERY CONTRACT

Objective

Measure acquisition, activation, purchase and retention without leaking document data.

Scope

- GA4, first-party product events, consent mode where applicable, attribution and server-side purchase confirmation.
- Search Console ownership/runbook, campaign parameters and internal business KPI definitions.
- Event catalog, identity rules, retention and privacy controls.

Implementation Requirements

- Define events before instrumentation: landing_view, tool_start, tool_complete, signup, document_preview, checkout_start, purchase_confirmed, document_download, email_sent, return_session.
- Use stable pseudonymous IDs; exclude names, addresses, tax identifiers, shipment lines, document text and email addresses.
- Deduplicate purchase events using order ID and mark test traffic.
- Create funnel and cohort queries with source-of-truth ownership.

Dependencies

- Tasks 02, 04 and 06.

Acceptance Criteria

- Event dictionary states trigger, properties, owner and privacy classification.
- GA4 receives only consent-eligible events; core functionality works when consent is denied.
- Purchases reconcile with billing data.
- Dashboards answer traffic-to-paid and repeat-use questions by landing page.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Analytics payload unit tests; consent denied/granted E2E; purchase deduplication; PII scanning of event payloads.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Treat analytics schema as a public contract and version breaking changes.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 09 - SEO platform and indexation controls

DELIVERY CONTRACT

Objective

Make useful tools and pages crawlable, canonical and measurable without generating thin content.

Scope

- Metadata, canonical URLs, robots directives, XML sitemap index, breadcrumbs, OpenGraph, structured data, redirects and 404/410 behavior.
- Google Search Console verification/runbook and IndexNow key/endpoint/submission queue.
- Content models for product, tool, template, guide, comparison, glossary and help pages.

Implementation Requirements

- Render meaningful public content server-side; do not expose private workspace URLs.
- Use only schema types supported by visible content; never manufacture review ratings.
- Submit IndexNow only for changed eligible URLs, deduplicated and rate-controlled.
- Prevent parameter, filter, auth, preview and duplicate pages from indexation.

Dependencies

- Tasks 01-03 and 08.

Acceptance Criteria

- Canonical, robots, sitemap and structured data validate across page types.
- No private/customer URL appears in sitemaps or search previews.
- Each indexed landing page provides a genuinely usable tool/template or decision-support value.
- Search Console and IndexNow setup has a repeatable production checklist.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Crawler test over built routes; structured-data validation fixtures; canonical/sitemap snapshots; noindex and redirect E2E.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Do not mass-produce AI pages. Ship only pages with unique intent, verified facts, useful interaction and editorial ownership.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 10 - CI/CD, release safety and supply chain

DELIVERY CONTRACT

Objective

Create repeatable releases with evidence and safe rollback.

Scope

- GitHub CI, Vercel previews, protected production deployment, migrations, E2E, dependency and secret scanning.
- Preview smoke tests, production promotion, rollback, release notes and post-deploy verification.
- SBOM/dependency inventory and third-party update policy.

Implementation Requirements

- CI gates: install lockfile, format check, lint, typecheck, unit/integration tests, build, migration validation and targeted E2E.
- Never run destructive migration automatically without backup and explicit rollout/rollback notes.
- Use preview-safe services and seeded synthetic data.
- Retain build/test evidence and associate Sentry releases with commits.

Dependencies

- Tasks 01-09.

Acceptance Criteria

- A clean pull request produces a preview and all evidence.
- A failing gate cannot promote to production.
- Rollback and forward-fix are rehearsed in staging.
- Production smoke test verifies auth, generation, checkout sandbox/probe and monitoring.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- CI self-test with intentional failures; migration backward/forward compatibility; preview isolation; rollback drill.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Commit workflow changes separately and document every required repository/Vercel control that cannot be automated.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 11 - TradeDocs domain model and database

DELIVERY CONTRACT

Objective

Model reusable trade master data, shipments and immutable document snapshots.

Scope

- Organizations, companies, contacts, customers/buyers, suppliers, addresses, products, variants/SKUs, units, currencies, trade terms, shipments, shipment items, packages, legs, document sets, documents, attachments and audit events.
- Draft/final/superseded/voided/archived states and numbering sequences.
- Snapshot/reference rules for reusable data versus issued documents.

Implementation Requirements

- Use normalized master data with explicit immutable snapshots per generated document.
- Support multiple addresses and roles; preserve country codes, units and currency explicitly.
- Use organization-scoped unique numbering with transactional allocation.
- Provide indexed search and cursor pagination without leaking tenant data.

Dependencies

- Common Tasks 01-05.

Acceptance Criteria

- Changing a product/customer never mutates a finalized document.
- A shipment can contain many items/packages and generate a coherent document set.
- Number allocation cannot duplicate under concurrency.
- RLS covers all tables, functions, attachments and derived views.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- ERD/constraints; concurrent numbering; snapshot immutability; unit/currency precision; RLS and pagination.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Produce a field-level dictionary and state-transition table before migrations.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 12 - Reusable company, customer and product workspace

DELIVERY CONTRACT

Objective

Make once-entered master data safely reusable across shipments and documents.

Scope

- Company profiles/branding, contacts, customers, suppliers, addresses, products/SKUs, descriptions, origin, HS-code field, units, weights, dimensions and pricing defaults.
- Import/export CSV, deduplication suggestions, archive/restore and change history.
- Data quality indicators and optional fields by workflow.

Implementation Requirements

- Never claim an HS code or country of origin is verified unless an approved verification service/process exists.
- CSV import requires preview, row-level errors, idempotency and rollback.
- Normalize search but preserve user-entered legal spelling in snapshots.
- Attachments use allowlisted MIME/content inspection, size limits and private storage.

Dependencies

- Trade Task 11 and common identity/security.

Acceptance Criteria

- A user enters each entity once and can select it in later shipments.
- Imports cannot partially corrupt master data; errors are downloadable.
- Duplicates can be merged only with explicit preview of downstream impact.
- Archived records remain visible on historical documents.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- CSV encoding/duplicates/invalid rows; merge/archive; malicious upload; two-tenant same-name isolation.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use realistic international addresses and long multilingual product descriptions in QA.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 13 - Shipment workspace and document dependency graph

DELIVERY CONTRACT

Objective

Center the product on a shipment record that feeds all related documents.

Scope

- Shipment creation, parties/roles, origin/destination, transport details, dates, currency, Incoterms/version, items, packages, marks, references, notes and status.
- Clone shipment/order, draft autosave, validation summary, activity timeline and document generation actions.
- Dependency graph that marks documents stale when source fields change.

Implementation Requirements

- Define which shipment fields feed each document and whether changes require regeneration.
- Use optimistic concurrency or version checks to prevent silent overwrite.
- Provide totals for quantities, packages, net/gross weight, volume and value with explicit units.
- Show Incoterms as user-selected commercial terms, not legal advice or automatic suitability.

Dependencies

- Trade Tasks 11-12.

Acceptance Criteria

- One shipment reliably populates every supported document.
- Changes show exactly which finalized/draft documents are stale and why.
- Clone creates a new identity/numbering history without linking mutable snapshots.
- Concurrent editing produces a conflict workflow, not lost updates.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Dependency-map unit tests; concurrency; clone; totals/reconciliation; stale/regenerate E2E.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Document the dependency graph in a machine-readable registry shared by UI, tests and generation.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 14 - Trade document rules and canonical schemas

DELIVERY CONTRACT

Objective

Define validated schemas and limitations for each supported document family.

Scope

- Commercial Invoice, Proforma Invoice, Packing List, Purchase Order, Quotation, Sales Order, Certificate of Origin template, Delivery Note and BOL/shipping-style document.
- Required/optional fields, numbering, totals, parties, signatures, status transitions and conversion relationships.
- Jurisdiction/carrier/chamber/customs limitation registry.

Implementation Requirements

- Create canonical schema per document type plus version and validator.
- Certificate of Origin is a preparation template unless endorsed by the competent body.
- BOL-style output must not claim carrier issuance, receipt, title, negotiability or contract of carriage; name and watermark it accordingly unless an authorized carrier workflow is later built.
- Legal copy must explain that templates do not ensure customs or trade compliance.

Dependencies

- Trade Tasks 11-13 and regulatory task.

Acceptance Criteria

- Every field in every output traces to shipment/master input or an explicit document override.
- Invalid/incomplete documents cannot be finalized.
- Official/endorsement limitations appear at the point of use and in output where required.
- Schema versions preserve historical reproducibility.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Schema fixtures per document; required-field matrix; forbidden legal claims snapshot; conversion and override precedence.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Treat document names and legal effect as high-risk copy requiring registry review.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 15 - Commercial and sales document workflows

DELIVERY CONTRACT

Objective

Implement complete workflows for Commercial Invoice, Proforma, PO, Quotation and Sales Order.

Scope

- Create/edit/duplicate/preview/finalize/void/download/email; custom numbering; parties; issue/validity/due dates; currency; line items; discounts/tax/freight/insurance/other charges; terms, notes and signatures.
- Convert Quotation -> Sales Order -> Proforma/Commercial Invoice and PO where semantically valid.
- Totals and rounding trace.

Implementation Requirements

- Use decimal-safe arithmetic, minor-unit rules and currency-specific formatting.
- Conversions copy into a new document snapshot and preserve lineage.
- Distinguish tax fields entered by user from jurisdiction-validated tax calculation.
- Finalized documents are immutable; corrections create a revision/credit workflow if supported or a superseding document.

Dependencies

- Trade Tasks 13-14.

Acceptance Criteria

- Totals reconcile across all charges, currencies and rounding fixtures.
- Conversions preserve traceability and never overwrite source documents.
- Line-item tables handle long text and many rows.
- Unsupported tax/compliance behavior is not inferred.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Currency/rounding/property tests; conversion lineage; 100-line stress E2E; revision/void workflow.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Keep arithmetic in a domain package, never React components or templates.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 16 - Packing, delivery and shipping-style workflows

DELIVERY CONTRACT

Objective

Implement Packing List, Delivery Note and constrained BOL/shipping-style outputs.

Scope

- Packages, marks/numbers, dimensions, net/gross weights, volume, contents allocation, delivery acknowledgment fields and transport references.
- Create/edit/preview/finalize/void/download/email with shipment reuse.
- Non-negotiable draft/shipping instruction presentation for BOL-style output unless an authorized issuance process exists.

Implementation Requirements

- Support item-to-package allocation and reconciliation.
- All units are explicit and conversions preserve original values and rounding.
- BOL-style document carries prominent legal status and issuer field; it cannot simulate carrier signature/endorsement.
- Signature capture, if offered, records signer intent/time but is not marketed as regulated e-signature without validation.

Dependencies

- Trade Tasks 13-14.

Acceptance Criteria

- Package contents reconcile to shipment items; total weights/volume reconcile.
- Outputs remain legible for many packages and mixed units.
- Legal status appears in UI, PDF and email.
- Finalized delivery acknowledgments retain audit history.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Allocation invariants; unit conversions; multi-page stress; disclaimer persistence; signature authorization.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use 'shipping instructions' or 'BOL-style draft' terminology according to approved registry copy.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 17 - Certificate of Origin preparation workflow

DELIVERY CONTRACT

Objective

Provide a useful origin-data template without impersonating an issuing authority.

Scope

- Exporter/producer/consignee, goods, origin criteria field, destination, transport, declarations, supporting attachments and template output.
- Preparation status, review checklist, export for external endorsement and record of returned endorsed copy if uploaded.
- Country/chamber-specific availability controlled by registry.

Implementation Requirements

- Default output is clearly marked 'Preparation template - not an issued certificate'.
- Do not add seals, authority marks or simulated signatures.
- Store evidence attachments privately and scan uploads.
- Separate user-declared origin from externally endorsed status and capture issuer/reference only from user evidence.

Dependencies

- Trade Tasks 12-14 and regulatory registry.

Acceptance Criteria

- No unendorsed output can be mistaken for an official certificate.
- Status transition to externally endorsed requires reference/evidence and audit event.
- Unsupported country-specific claims are blocked.
- Output and UI explain the next external step.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Status/legal-label snapshots; attachment authorization/scanning; unsupported-country cases; audit transition tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Have legal copy approved in the registry before enabling this workflow in production.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 18 - Trade PDF engine, templates and branding

DELIVERY CONTRACT

Objective

Render every supported trade document and set as original, polished, reproducible PDFs.

Scope

- Original templates, logo/company branding, multi-page tables, repeated headers, totals, signatures, page numbers, IDs, timestamps and locale/currency/unit formatting.
- Preview, final PDF, secure storage/download, email delivery and regeneration.
- Renderer/template versioning and visual baseline library.

Implementation Requirements

- Canonical document schema is independent from renderer.
- Server-side jobs are idempotent, bounded and observable.
- Long descriptions and international characters wrap safely; tables repeat headers and prevent orphan totals.
- Legal labels are non-removable where required.

Dependencies

- Trade Tasks 14-17 and common billing/email.

Acceptance Criteria

- Each document passes short/typical/stress rendering with no clipping or overlap.
- A historical document regenerates under its original schema/template version.
- Final output hash and audit metadata are stored.
- Preview/final access follows entitlement and tenant policy.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Rendered visual regression for all types; 1/3/10-page fixtures; Unicode, logo, totals, disclaimer and retry tests.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Render and visually review every document type before accepting any baseline update.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 19 - Document sets, bulk operations and exports

DELIVERY CONTRACT

Objective

Turn individual documents into a cohesive shipment deliverable.

Scope

- Create named document sets from a shipment, choose versions, validate completeness, bulk generate, ZIP download and email/share.
- Set-level status, manifest, checksums, regeneration warnings and partial-failure recovery.
- CSV import/export for approved data; no unrequested Word/Excel claims.

Implementation Requirements

- Generate a manifest listing document type, number, version, hash and legal status.
- Stream or queue large jobs with progress; enforce quotas and expiration.
- ZIP paths and filenames are sanitized against traversal/collision.
- Partial failures are resumable and never silently omit a document.

Dependencies

- Trade Task 18 and common security/email/billing.

Acceptance Criteria

- A set is internally consistent and traceable to one shipment revision.
- User sees success/failure per document and can retry safely.
- ZIP contains the manifest and exactly the selected outputs.
- Large sets do not exceed serverless limits without a queued path.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- ZIP slip/collision; partial job retry; quota/concurrency; manifest/hash verification; expired share links.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Do not mark a set complete unless every selected artifact passed generation and checksum verification.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 20 - TradeDocs dashboard, history and collaboration

DELIVERY CONTRACT

Objective

Provide a durable B2B workspace for repeated shipment activity.

Scope

- Dashboard, recent shipments/documents, customers/products, drafts, document sets, search/filter/sort, duplicate/clone, archive, billing and settings.
- Organization roles, invitations, activity history and ownership-sensitive actions.
- Customer/product/shipment history and related-document navigation.

Implementation Requirements

- Use server-authorized cursor pagination and saved filters.
- Final records remain immutable; edits create revisions.
- Collaboration conflicts use version checks and meaningful resolution UI.
- Audit who generated, finalized, voided, downloaded, emailed or externally endorsed each artifact.

Dependencies

- Trade Tasks 11-19 and common identity.

Acceptance Criteria

- A returning user can create a new shipment mainly from reused data.
- Users navigate from entity to all related shipments/documents without leakage.
- Role limits are consistent in UI and server APIs.
- Mobile and desktop loading/empty/error states are complete.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Multi-member RBAC; search/history; concurrent edit; archive; full returning-customer journey.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Test real retention flow: second shipment, reused customer/products, set generation and history retrieval.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 21 - Trade calculators, free generators and acquisition

DELIVERY CONTRACT

Objective

Build useful anonymous acquisition tools that lead naturally to saved workspace value.

Scope

- CBM calculator, dimensional-weight calculator, bounded landed-cost estimator, Incoterms explainer/selector, and free Commercial Invoice, Packing List and Proforma generators.
- Anonymous draft lifecycle, abuse limits, optional account conversion and reuse.
- Clear methodology, units, assumptions, source/review metadata and limitations.

Implementation Requirements

- Calculators are deterministic domain modules with formulas and units documented.
- Landed cost never presents duty/tax as authoritative when tariff/classification data is not validated; show user-provided rate or range assumptions.
- Anonymous data expires quickly and is not indexed/logged.
- Saving claims the draft into the authenticated tenant using a one-time token.

Dependencies

- Common SEO/analytics/security and Trade document/calculation tasks.

Acceptance Criteria

- Tools work without account under safe quotas and do not leak draft URLs.
- Every result shows units, formula/assumptions and limitations.
- Conversion to account preserves the draft exactly once.
- Tool completion and upgrade events contain no trade data.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Formula/property/unit tests; anonymous expiry and takeover protection; rate limit; conversion idempotency; SEO crawl.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Do not add a country-specific compliance calculator without primary-source rules and registry approval.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 22 - TradeDocs SEO and editorial system

DELIVERY CONTRACT

Objective

Build search visibility around document intent, shipment workflow and trusted educational value.

Scope

- Pages for each document/template/generator/calculator; workflow hubs; Incoterms topics; glossary; guides; help; pricing; security/legal/trust pages.
- Editorial metadata, reviewer, last reviewed, source references and update queue.
- Internal links from free tool -> relevant document set/workspace and from guides -> tools.

Implementation Requirements

- Avoid thin country/port/HS-code pages and unsupported regulatory assertions.
- Visible content must match structured data and actual product capability.
- Do not index private shipment previews, shared expiring URLs, filter states or duplicate templates.
- Create an editorial QA checklist for factual, legal and commercial claims.

Dependencies

- Common SEO/analytics and Trade free tools/regulatory.

Acceptance Criteria

- Each indexed page serves unique intent and provides a functioning tool/template or substantive guide.
- Canonical/sitemap/robots prevent index bloat.
- Factual pages display review/source metadata.
- Funnels segment by landing/tool/document intent.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Full-route crawler; structured data; canonicals/robots; broken links; factual metadata completeness; Core Web Vitals budgets.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Never generate pages solely from keyword lists; require product value and editorial ownership.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 23 - TradeDocs admin, support and operational controls

DELIVERY CONTRACT

Objective

Enable safe customer support, document-job operations and commercial administration.

Scope

- Tenant/customer lookup, shipment/document metadata, job/retry queue, order/refund/entitlement view, abuse controls, support notes and content/regulatory approvals.
- Admin roles, reauthentication, masking, reason codes, audit and break-glass procedure.
- Operational dashboards for generation, set jobs, webhooks, email and storage.

Implementation Requirements

- Default to metadata and masked values; content access requires scoped reason and audit.
- Never allow silent impersonation.
- Bulk/retry actions are idempotent and tenant scoped.
- Admin pages are noindex, outside public navigation and protected server-side.

Dependencies

- All prior Trade tasks and common security/billing.

Acceptance Criteria

- Normal support cases require no direct database access.
- Every privileged action is attributable and reviewable.
- Role/restriction controls cannot be bypassed through APIs.
- Job retry does not duplicate documents, sets, payments or emails.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Admin RBAC; mask/reveal; retry idempotency; restriction bypass; audit completeness; refund reconciliation.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use only synthetic shipments and documents in admin QA.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 24 - Trade regulatory, terminology and source registry

DELIVERY CONTRACT

Objective

Maintain legal limitations, trade terminology and high-risk workflow decisions over time.

Scope

- Registry for Incoterms edition/licensing notes, customs/trade authority guidance, origin documentation, sanctions/export-control disclaimers, document-specific legal status and supported jurisdictions.
- Monitoring, editorial review, incident correction, feature flags and customer notice workflow.
- Monthly backstop review plus triggered review for authoritative changes.

Implementation Requirements

- Record source, authority, jurisdiction, effective interval, retrieval date, reviewer, affected schemas/templates/copy and approval.
- Alerts create human review tasks and never autonomously change legal behavior.
- Sanctions/classification checks are not claimed unless an approved data provider and workflow are implemented.
- Retire or disable misleading functionality immediately via flag when risk is discovered.

Dependencies

- Trade Tasks 14-17, admin, CI.

Acceptance Criteria

- Every high-risk label/claim maps to approved registry text.
- Certificate/BOL limitations remain present across UI/PDF/email.
- A registry change identifies affected documents/pages/tests and rollout owner.
- Quarterly workflow review and monthly source review are evidenced.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Registry validation/staleness; legal-copy snapshots; feature-flag disable; source-change drill; affected-content query.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Use counsel/qualified reviewer checkpoints as explicit blockers where the product could imply official issuance.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Task 25 - TradeDocs launch, monitoring and post-launch optimization

DELIVERY CONTRACT

Objective

Launch a production service that can sell, recover and improve from evidence.

Scope

- Production setup, DNS/email, legal policies, payment activation, backups, restore, monitoring, support/on-call and incident response.
- Synthetic shipment -> set -> purchase -> PDF -> email -> later retrieval journey.
- SEO/indexation, acquisition funnels, activation/retention cohorts, support and document-type adoption.

Implementation Requirements

- Alert on generation/set failures, stale job queues, payment/webhook mismatch, storage growth, email failure, permission anomalies and abuse.
- Define SLOs, runbooks, owners, escalation and rollback triggers.
- Perform security/privacy/accessibility/regulatory and backup restore sign-offs.
- Prioritize post-launch work by SEO traction, revenue, conversion, repeat usage, expansion potential and support burden.

Dependencies

- All common and Trade tasks.

Acceptance Criteria

- Production-like user completes a paid coherent document set with secure delivery and history.
- Backup restore and rollback drills succeed.
- GA4/product/billing metrics reconcile and Search Console/IndexNow are operational.
- No P0/P1, cross-tenant leak or unresolved official-document ambiguity remains.

Required Tests

- Run format, lint, strict TypeScript typecheck, unit tests, affected integration tests, and production build with zero unexplained warnings.
- Add Playwright coverage for the primary happy path, validation failure, authorization failure, retry/reload behavior, and mobile viewport.
- Add accessibility checks for affected pages and prove tenant isolation for every new data path.
- Full dress rehearsal; production smoke; synthetic monitoring; restore/rollback; incident/support tabletop; load test for bulk generation.

AI Agent Instructions

- Read Master Instructions, the dependency tasks, and all affected living documents before editing code.
- Show an implementation checklist mapped one-to-one to acceptance criteria; keep evidence in the task report.
- After implementation, run the required gates, inspect the real UI/output, update documentation and CHANGELOG, then create a focused conventional commit. Do not proceed with failures.
- Deliver a signed launch evidence report, operational ownership matrix and unresolved-risk register before go-live.

Completion evidence: link the commit, test output, screenshots/rendered samples, migration notes, updated documents and remaining risks in the task report.

Release Gate Checklist

Product

- [ ] All task acceptance criteria are evidenced.
- [ ] Pricing, refund, entitlement and support policies are approved.
- [ ] No placeholder copy, dead route, fake testimonial or unfinished state is public.

Engineering

- [ ] Clean production build; migrations applied safely; no critical dependency findings.
- [ ] Unit, integration, E2E, accessibility, PDF visual and load tests pass.
- [ ] Feature flags, rollback and data compatibility are proven.

Security and privacy

- [ ] Threat model and RLS matrix reviewed; no cross-tenant finding remains.
- [ ] Secrets, logs, uploads, signed links, admin controls, retention/export/delete are verified.
- [ ] Turnstile and rate limits operate without blocking legitimate recovery paths.

Commercial

- [ ] Verified webhook grants entitlement; refunds/disputes reconcile; receipt/email delivery works.
- [ ] Purchase funnel and server-confirmed revenue events reconcile.
- [ ] Support can locate and resolve an order without database editing.

Operations

- [ ] Sentry releases, alerts, health checks, dashboards, on-call contacts and runbooks are live.
- [ ] Backup and restore completed; RPO/RTO and incident process approved.
- [ ] Synthetic monitoring and post-deploy smoke checks run from production.

SEO and regulatory

- [ ] Canonical/robots/sitemap/schema validate; Search Console and IndexNow are configured.
- [ ] All high-risk rules and claims map to approved primary-source registry entries.
- [ ] Editorial and regulatory review cadence, owners and alert workflow are scheduled.

Post-Launch Operating Loop

Observe

- Daily: uptime, errors, queues, payment/webhook reconciliation, delivery, abuse and support.
- Weekly: acquisition funnel, page/tool conversion, search queries, failed searches, repeat use and document/template adoption.
- Monthly: regulatory/source review, access review, backup evidence, dependency risk, SEO content decay and KPI prioritization.
- Quarterly: restore drill, incident tabletop, threat-model review, accessibility regression and vendor/subprocessor review.

Prioritize

- Score work using SEO traction, revenue, conversion rate, returning users, keyword/document expansion and support burden.
- Fix security, payment integrity, data loss and regulatory correctness before growth experiments.
- Use controlled experiments with predeclared metric, guardrail and stopping rule; do not change product randomly.

Maintain

- Add new tax year, jurisdiction, document type or provider only through versioned registry/schema/fixture changes.
- Preserve historical reproducibility and never rewrite issued/purchased artifacts in place.
- Keep CHANGELOG, source registry, runbooks and customer-facing last-reviewed dates synchronized with releases.

END OF SPECIFICATION